

13.4 A more nuanced IO type

If we use `TailRec` as our IO type, this solves the stack overflow problem, but the other two problems with the monad still remain—it's inexplicit about what kinds of effects may occur, and it has no concurrency mechanism or means of performing I/O without blocking the current thread of execution.

During execution, the run interpreter will look at a `TailRec` program such as `FlatMap(Suspend(s), k)`, in which case the next thing to do is to call `s()`. The program is returning control to run, requesting that it execute some effect `s`, wait for the result, and respond by passing the resulting value to `k` (which may subsequently return a further request). At the moment, the interpreter can't know anything about *what kind of effects* the program is going to have. It's completely opaque. So the only thing it can do is call `s()`. Not only can that have an arbitrary and unknowable side effect, there's no way that the interpreter could allow asynchronous calls if it wanted to. Since the suspension is a `Function0`, all we can do is call it and wait for it to complete.

What if we used `Par` from chapter 7 for the suspension instead of `Function0`? Let's call this type `Async`, since the interpreter can now support asynchronous execution.

Listing 13.5 Defining our `Async` type

```
sealed trait Async[A] {
  def flatMap[B](f: A => Async[B]): Async[B] =
    FlatMap(this, f)
  def map[B](f: A => B): Async[B] =
    flatMap(f andThen (Return(_)))
}
case class Return[A](a: A) extends Async[A]
case class Suspend[A](resume: Par[A]) extends Async[A]
case class FlatMap[A, B](sub: Async[A],
                        k: A => Async[B]) extends Async[B]
```

Note that the `resume` argument to `Suspend` is now a `Par[A]` rather than a `() => A` (or a `Function0[A]`). The implementation of `run` changes accordingly—it now returns a `Par[A]` rather than an `A`, and we rely on a separate tail-recursive step function to reassociate the `FlatMap` constructors:

```
@annotation.tailrec
def step[A](async: Async[A]): Async[A] = async match {
  case FlatMap(FlatMap(x, f), g) => step(x flatMap (a => f(a) flatMap g))
  case FlatMap(Return(x), f) => step(f(x))
  case _ => async
}

def run[A](async: Async[A]): Par[A] = step(async) match {
  case Return(a) => Par.unit(a)
  case Suspend(r) => Par.flatMap(r)(a => run(a))
  case FlatMap(x, f) => x match {
    case Suspend(r) => Par.flatMap(r)(a => run(f(a)))
    case _ => sys.error("Impossible; `step` eliminates these cases")
  }
}
```